In [2]:
```
# LIBRARIES
import pandas as pd
import matplotlib.pyplot as plt
```

ECOLI (support >= 3, freq >= 70%)stmp matches

In [2]:
```
base_path = r"C:\VSC\bioinf\ecoli"

solution_file = base_path + r"\data\variants.csv"
truth_file = base_path + r"\stmpileup_var.csv"

cols = ["operation", "position", "base"]

solution = pd.read_csv(solution_file, header=None, names=cols)
truth = pd.read_csv(truth_file, header=None, names=cols)

solution["key"] = (
    solution["position"].astype(str) + "_" +
    solution["operation"] + "_" +
    solution["base"]
)

truth["key"] = (
    truth["position"].astype(str) + "_" +
    truth["operation"] + "_" +
    truth["base"]
)

solution_keys = set(solution["key"])
truth_keys = set(truth["key"])

matches = solution_keys.intersection(truth_keys)

num_matches = len(matches)
total_truth = len(truth_keys)

coverage = num_matches / total_truth * 100 if total_truth else 0

print(f"E-coli <> samtools coverage\n")
print(f"Matches found    : {num_matches}")
print(f"stmp variants    : {total_truth}")
print(f"Coverage of stmp : {coverage:.2f}%")
```

```
E-coli <> samtools coverage

Matches found    : 14511
stmp variants    : 14514
Coverage of stmp : 99.98%
```

stmp metrics

In [3]:
```
lookup = {}

for idx, row in solution.iterrows():

    key = (row["operation"], row["base"], row["position"])
```

```
    if key not in lookup:
        lookup[key] = []

    lookup[key].append(idx)

used_solution_indices = set()

tp = 0  # true positives
fn = 0  # false negatives

for _, trow in truth.iterrows():

    found = False

    t_op = trow["operation"]
    t_pos = trow["position"]
    t_base = trow["base"]

    key = (t_op, t_base, t_pos)

    if key in lookup:

        for s_idx in lookup[key]:

            if s_idx not in used_solution_indices:
                tp += 1
                used_solution_indices.add(s_idx)
                found = True
                break

    if not found:
        fn += 1

fp = len(solution) - len(used_solution_indices)

precision = tp / (tp + fp) if (tp + fp) > 0 else 0
recall = tp / (tp + fn) if (tp + fn) > 0 else 0
f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else

total = tp + fp + fn
accuracy = tp / total if total > 0 else 0

# convert to percentages
precision *= 100
recall *= 100
f1 *= 100
accuracy *= 100

print(f"E-coli <> samtools metrics\n")

print("\n================ CONFUSION MATRIX ================\n")

print("                    Truth")
print("                Present   Absent")
print(f"Pred Present    {tp:<8}  {fp}")
print(f"Pred Absent     {fn:<8}  -")
```

```
print("\n================ STATS ================\n")

print(f"Accuracy  : {accuracy:.2f}%")
print(f"Precision : {precision:.2f}%")
print(f"Recall    : {recall:.2f}%")
print(f"F1 Score  : {f1:.2f}%")
```

```
E-coli <> samtools metrics


================ CONFUSION MATRIX ================

                   Truth
               Present   Absent
Pred Present   14511     3653
Pred Absent    3         -

================ STATS ================

Accuracy  : 79.88%
Precision : 79.89%
Recall    : 99.98%
F1 Score  : 88.81%
```

ground truth matches

In [4]:
```
solution_file = r"C:\VSC\bioinf\ecoli\data\variants.csv"
truth_file = r"C:\VSC\bioinf\ecoli\data\ecoli_mutated.csv"

cols = ["operation", "position", "base"]

solution = pd.read_csv(solution_file, header=None, names=cols)
truth = pd.read_csv(truth_file, header=None, names=cols)

solution["key"] = (
    solution["position"].astype(str) + "_" +
    solution["operation"] + "_" +
    solution["base"]
)

truth["key"] = (
    truth["position"].astype(str) + "_" +
    truth["operation"] + "_" +
    truth["base"]
)

solution_keys = set(solution["key"])
truth_keys = set(truth["key"])

matches = solution_keys.intersection(truth_keys)

num_matches = len(matches)
total_truth = len(truth_keys)

coverage = num_matches / total_truth * 100 if total_truth else 0
```

```
print(f"E-coli <> ground truth coverage\n")
print(f"Matches found      : {num_matches}")
print(f"Ground truth       : {total_truth}")
print(f"Coverage of ground : {coverage:.2f}%")
```

```
E-coli <> ground truth coverage

Matches found      : 15284
Ground truth       : 19494
Coverage of ground : 78.40%
```

ground truth + off-by-one matches

In [5]:
```
lookup = {}

for idx, row in solution.iterrows():

    key = (row["operation"], row["base"], row["position"])

    if key not in lookup:
        lookup[key] = []

    lookup[key].append(idx)

exact_matches = 0
off_by_one_matches = 0

used_solution_indices = set()

for _, trow in truth.iterrows():

    t_op = trow["operation"]
    t_pos = trow["position"]
    t_base = trow["base"]

    found = False

    exact_key = (t_op, t_base, t_pos)

    if exact_key in lookup:

        for s_idx in lookup[exact_key]:

            if s_idx not in used_solution_indices:
                exact_matches += 1
                used_solution_indices.add(s_idx)
                found = True
                break

    if not found:

        for offset in (-1, 1):

            off_key = (t_op, t_base, t_pos + offset)

            if off_key in lookup:
```

```
                for s_idx in lookup[off_key]:

                    if s_idx not in used_solution_indices:
                        off_by_one_matches += 1
                        used_solution_indices.add(s_idx)
                        found = True
                        break

            if found:
                break

total_matched = exact_matches + off_by_one_matches
coverage = (total_matched / len(truth) * 100) if len(truth) else 0

print(f"E-coli + off-by-one matches <> ground truth coverage\n")
print(f"Exact matches        : {exact_matches}")
print(f"Off-by-one matches   : {off_by_one_matches}")
print(f"Total matched        : {total_matched}")
print(f"Ground truth total   : {len(truth)}")
print(f"Coverage             : {coverage:.2f}%")
```

```
E-coli + off-by-one matches <> ground truth coverage

Exact matches        : 15284
Off-by-one matches   : 2283
Total matched        : 17567
Ground truth total   : 19494
Coverage             : 90.11%
```

ground truth + off-by-one metrics

In [6]:

```
lookup = {}

for idx, row in solution.iterrows():

    key = (row["operation"], row["base"], row["position"])

    if key not in lookup:
        lookup[key] = []

    lookup[key].append(idx)

used_solution_indices = set()

tp = 0  # true positives
fn = 0  # false negatives

for _, trow in truth.iterrows():

    found = False

    t_op = trow["operation"]
    t_pos = trow["position"]
    t_base = trow["base"]

    positions_to_check = [t_pos, t_pos - 1, t_pos + 1]
```

```
    for pos in positions_to_check:

        key = (t_op, t_base, pos)

        if key in lookup:

            for s_idx in lookup[key]:

                if s_idx not in used_solution_indices:
                    tp += 1
                    used_solution_indices.add(s_idx)
                    found = True
                    break

        if found:
            break

    if not found:
        fn += 1

fp = len(solution) - len(used_solution_indices)

precision = tp / (tp + fp) if (tp + fp) > 0 else 0
recall = tp / (tp + fn) if (tp + fn) > 0 else 0
f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else

total = tp + fp + fn
accuracy = tp / total if total > 0 else 0

# convert to percentages
precision *= 100
recall *= 100
f1 *= 100
accuracy *= 100

print(f"E-coli + off-by-one matches <> ground truth metrics\n")

print("\n================ CONFUSION MATRIX ================\n")

print("                    Truth")
print("                Present   Absent")
print(f"Pred Present    {tp:<8}  {fp}")
print(f"Pred Absent     {fn:<8}  -")

print("\n================ STATS ================\n")

print(f"Accuracy  : {accuracy:.2f}%")
print(f"Precision : {precision:.2f}%")
print(f"Recall    : {recall:.2f}%")
print(f"F1 Score  : {f1:.2f}%")
```

```
E-coli + off-by-one matches <> ground truth metrics


================ CONFUSION MATRIX ================

                   Truth
                Present   Absent
Pred Present    17567     597
Pred Absent     1927      -

================ STATS ================

Accuracy  : 87.44%
Precision : 96.71%
Recall    : 90.11%
F1 Score  : 93.30%
```

stmp groups

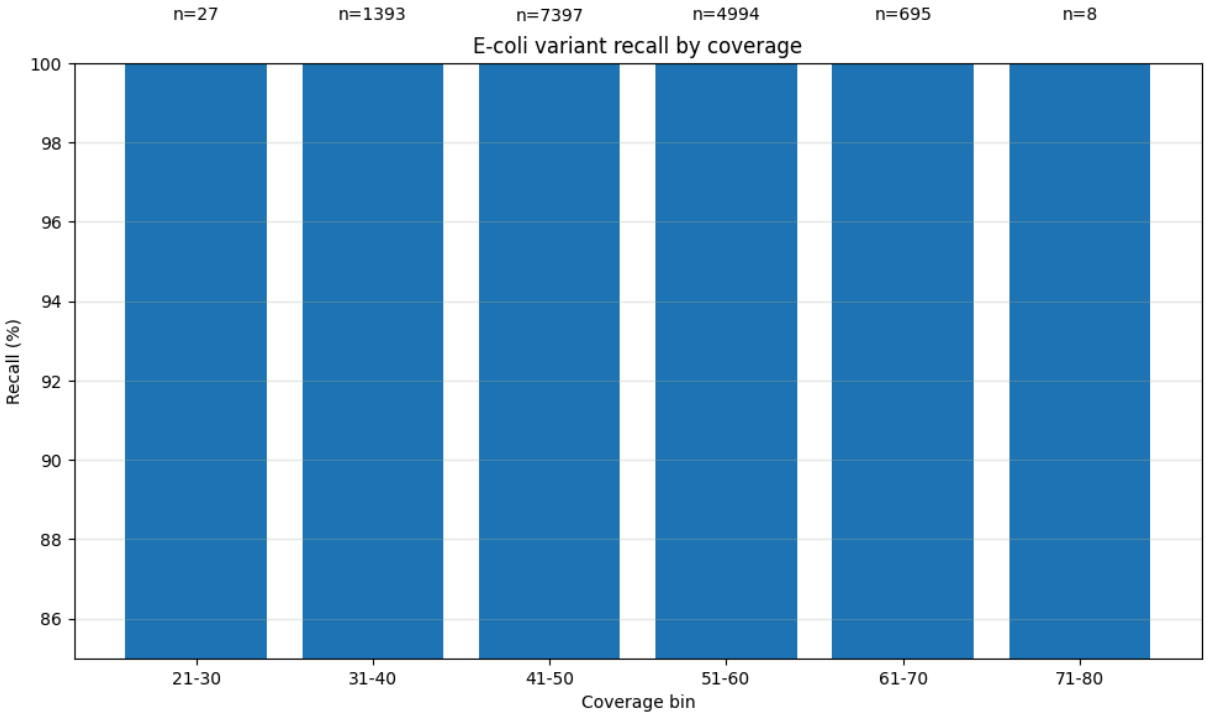
In [7]:

```python
base_path = r"C:\VSC\bioinf\ecoli"

pileup_file = base_path + r"\pileup.txt"
solution_file = base_path + r"\data\variants.csv"
truth_file = base_path + r"\stmpileup_var.csv"

cols = ["operation", "position", "base"]

# ============================================================
# LOAD COVERAGE FROM PILEUP
# ============================================================

coverage_dict = {}

with open(pileup_file) as f:
    for line in f:
        fields = line.strip().split()

        if len(fields) < 4:
            continue

        pos = int(fields[1]) - 1   # mpileup je 1-based, CSV je 0-based
        cov = int(fields[3])

        coverage_dict[pos] = cov

# ============================================================
# LOAD VARIANT FILES
# ============================================================

solution = pd.read_csv(solution_file, header=None, names=cols)
truth = pd.read_csv(truth_file, header=None, names=cols)

# ============================================================
# CREATE KEYS
# ============================================================

solution["key"] = (
```

```
    solution["position"].astype(str) + "_" +
    solution["operation"] + "_" +
    solution["base"]
)

truth["key"] = (
    truth["position"].astype(str) + "_" +
    truth["operation"] + "_" +
    truth["base"]
)

solution_keys = set(solution["key"])

# ============================================================
# MATCH TRUTH VARIANTS WITH COVERAGE
# ============================================================

records = []

for _, row in truth.iterrows():

    pos = int(row["position"])
    cov = coverage_dict.get(pos, 0)

    matched = row["key"] in solution_keys

    records.append({
        "position": pos,
        "coverage": cov,
        "matched": int(matched)
    })

results = pd.DataFrame(records)

# ============================================================
# COVERAGE BINS
# ============================================================

bins = [20, 30, 40, 50, 60, 70, 80]

labels = [
    "21-30",
    "31-40",
    "41-50",
    "51-60",
    "61-70",
    "71-80"
]

results["coverage_bin"] = pd.cut(
    results["coverage"],
    bins=bins,
    labels=labels,
    include_lowest=True
)
```

```
# ============================================================
# STATISTICS BY COVERAGE BIN
# ============================================================

stats = (
    results
    .groupby("coverage_bin", observed=False)
    .agg(
        total=("matched", "count"),
        matches=("matched", "sum")
    )
    .reset_index()
)

stats["recall"] = stats["matches"] / stats["total"] * 100
stats["recall"] = stats["recall"].fillna(0)

print(stats)

# ============================================================
# PLOT
# ============================================================

plt.figure(figsize=(10, 6))

bars = plt.bar(
    stats["coverage_bin"].astype(str),
    stats["recall"]
)

for bar, total in zip(bars, stats["total"]):

    height = bar.get_height()

    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height + 1,
        f"n={total}",
        ha="center",
        va="bottom"
    )

plt.xlabel("Coverage bin")
plt.ylabel("Recall (%)")
plt.title("E-coli variant recall by coverage")

plt.ylim(85, 100)
plt.grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.show()
```

	coverage_bin	total	matches	recall
0	21-30	27	27	100.000000
1	31-40	1393	1393	100.000000
2	41-50	7397	7394	99.959443
3	51-60	4994	4994	100.000000
4	61-70	695	695	100.000000
5	71-80	8	8	100.000000



stmp stats by mutation type

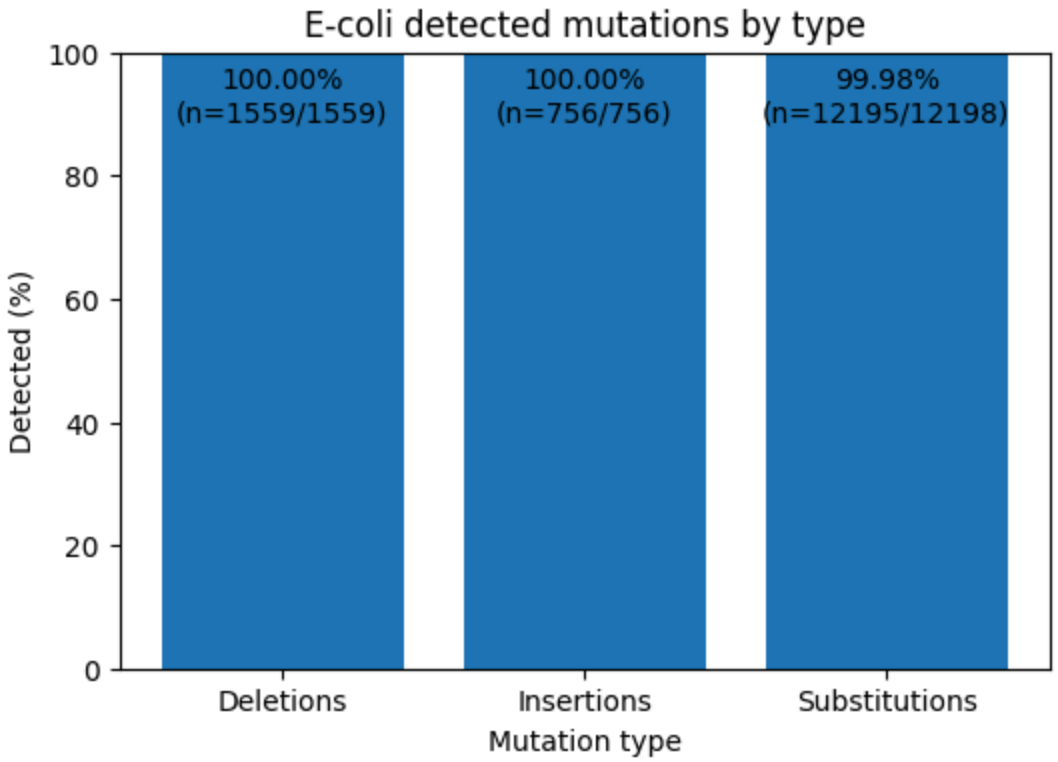
In [26]:
```
base_path = r"C:\VSC\bioinf\ecoli"

truth = pd.read_csv(base_path + r"\stmpileup_var.csv")
solution = pd.read_csv(base_path + r"\data\variants.csv")

truth.columns = ["operation", "position", "base"]
solution.columns = ["operation", "position", "base"]

# CLEAN DATA TYPES

truth["operation"] = truth["operation"].astype(str)
truth["position"] = truth["position"].astype(int)
truth["base"] = truth["base"].astype(str)

solution["operation"] = solution["operation"].astype(str)
solution["position"] = solution["position"].astype(int)
solution["base"] = solution["base"].astype(str)

# BUILD LOOKUP TABLE FOR SOLUTION VARIANTS

solution_set = set(
    zip(
        solution["operation"],
        solution["position"],
        solution["base"]
```

```
    )
)

# ANALYZE DETECTED MUTATIONS BY TYPE

results = []

for mut_type in sorted(truth["operation"].unique()):

    subset = truth[truth["operation"] == mut_type]

    detected = 0
    total = len(subset)

    for _, row in subset.iterrows():

        key = (
            row["operation"],
            row["position"],
            row["base"]
        )

        if key in solution_set:
            detected += 1

    detected_percent = detected / total * 100 if total > 0 else 0

    results.append({
        "mutation_type": mut_type,
        "detected": detected,
        "total": total,
        "detected_percent": detected_percent
    })

df_results = pd.DataFrame(results)

# ADD READABLE MUTATION NAMES

type_names = {
    "X": "Substitutions",
    "I": "Insertions",
    "D": "Deletions"
}

df_results["mutation_name"] = df_results["mutation_type"].map(type_names)

df_results = df_results[
    [
        "mutation_name",
        "mutation_type",
        "detected",
        "total",
        "detected_percent"
    ]
]
```

```
df_results["detected_percent"] = df_results["detected_percent"].round(2)

# PRINT RESULTS

print("=== DETECTION BY MUTATION TYPE ===\n")

for _, row in df_results.iterrows():
    print(
        f"{row['mutation_name']} ({row['mutation_type']}): "
        f"{row['detected']}/{row['total']} detected "
        f"({row['detected_percent']:.2f}%)"
    )

print("\nResults table:")
display(df_results)

# PLOT RESULTS

plt.figure(figsize=(6, 4))

plt.bar(
    df_results["mutation_name"],
    df_results["detected_percent"]
)

for i, row in df_results.iterrows():
    value = row["detected_percent"]
    detected = row["detected"]
    total = row["total"]

    plt.text(
        i,
        value - 12,
        f"{value:.2f}%\n(n={detected}/{total})",
        ha="center",
        va="bottom"
    )

plt.ylabel("Detected (%)")
plt.xlabel("Mutation type")
plt.title("E-coli detected mutations by type")
plt.ylim(0, 100)

plt.show()
```

```
=== DETECTION BY MUTATION TYPE ===

Deletions (D): 1559/1559 detected (100.00%)
Insertions (I): 756/756 detected (100.00%)
Substitutions (X): 12195/12198 detected (99.98%)

Results table:
```

	mutation_name	mutation_type	detected	total	detected_percent
0	Deletions	D	1559	1559	100.00
1	Insertions	I	756	756	100.00
2	Substitutions	X	12195	12198	99.98



stmp stats by mutation type (INSERTIONS ONE OFF)

In [10]:
```
base_path = r"C:\VSC\bioinf\ecoli"

truth = pd.read_csv(base_path + r"\data\ecoli_mutated.csv")
solution = pd.read_csv(base_path + r"\data\variants.csv")

truth.columns = ["operation", "position", "base"]
solution.columns = ["operation", "position", "base"]

# CLEAN DATA TYPES

truth["operation"] = truth["operation"].astype(str)
truth["position"] = truth["position"].astype(int)
truth["base"] = truth["base"].astype(str)

solution["operation"] = solution["operation"].astype(str)
solution["position"] = solution["position"].astype(int)
solution["base"] = solution["base"].astype(str)

# BUILD LOOKUP TABLE FOR SOLUTION VARIANTS

solution_set = set(
    zip(
        solution["operation"],
```

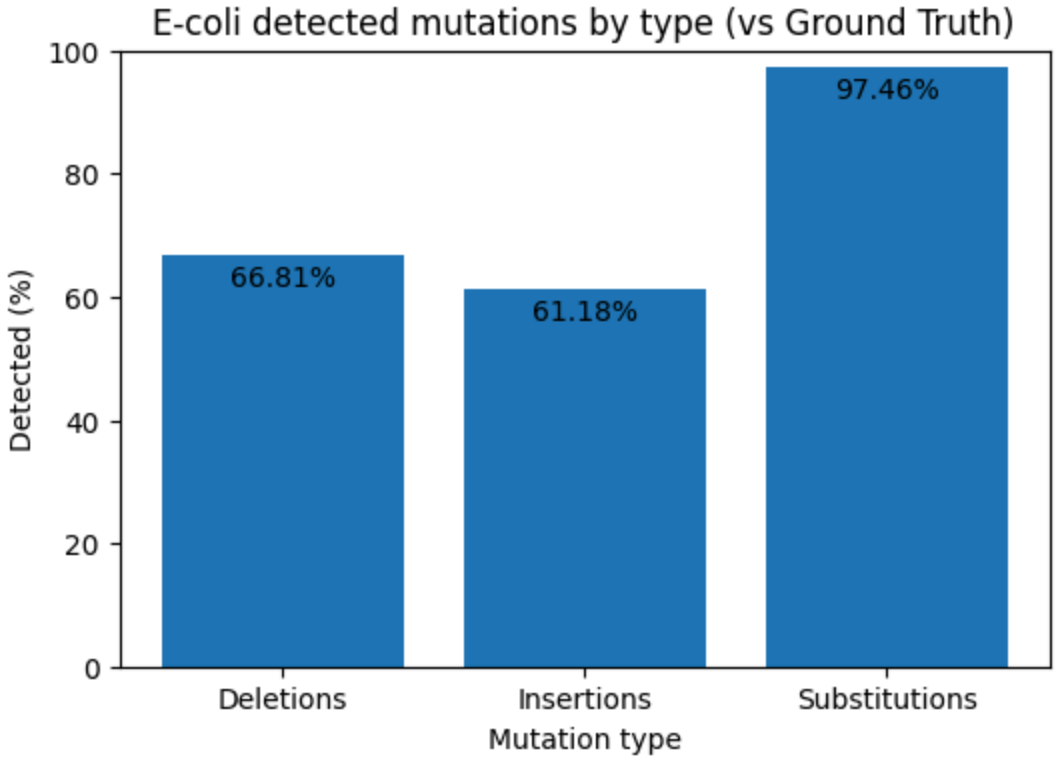
```
        solution["position"],
        solution["base"]
    )
)

# ANALYZE DETECTED MUTATIONS BY TYPE

results = []

for mut_type in sorted(truth["operation"].unique()):

    subset = truth[truth["operation"] == mut_type]

    detected = 0
    total = len(subset)

    for _, row in subset.iterrows():

        key = (
            row["operation"],
            row["position"],
            row["base"]
        )

        if row["operation"] == "I":
            key_minus_one = (
                row["operation"],
                row["position"] - 1,
                row["base"]
            )

            key_plus_one = (
                row["operation"],
                row["position"] + 1,
                row["base"]
            )

            if key in solution_set or key_minus_one in solution_set or key_plus_one
                detected += 1

        else:
            if key in solution_set:
                detected += 1

    detected_percent = detected / total * 100 if total > 0 else 0

    results.append({
        "mutation_type": mut_type,
        "detected": detected,
        "total": total,
        "detected_percent": detected_percent
    })

df_results = pd.DataFrame(results)

# ADD READABLE MUTATION NAMES
```

```
type_names = {
    "X": "Substitutions",
    "I": "Insertions",
    "D": "Deletions"
}

df_results["mutation_name"] = df_results["mutation_type"].map(type_names)

df_results = df_results[
    [
        "mutation_name",
        "mutation_type",
        "detected",
        "total",
        "detected_percent"
    ]
]

df_results["detected_percent"] = df_results["detected_percent"].round(2)

# PRINT RESULTS

print("=== DETECTION BY MUTATION TYPE ===\n")

for _, row in df_results.iterrows():
    print(
        f"{row['mutation_name']} ({row['mutation_type']}): "
        f"{row['detected']}/{row['total']} detected "
        f"({row['detected_percent']:.2f}%)"
    )

print("\nResults table:")
display(df_results)

# PLOT RESULTS

plt.figure(figsize=(6, 4))

plt.bar(
    df_results["mutation_name"],
    df_results["detected_percent"]
)

for i, value in enumerate(df_results["detected_percent"]):
    plt.text(
        i,
        value - 6,
        f"{value:.2f}%",
        ha="center",
        va="bottom"
    )

plt.ylabel("Detected (%)")
plt.xlabel("Mutation type")
plt.title("E-coli detected mutations by type (vs Ground Truth)")
```

```
plt.ylim(0, 100)

plt.show()
```

```
=== DETECTION BY MUTATION TYPE ===

Deletions (D): 1918/2871 detected (66.81%)
Insertions (I): 1779/2908 detected (61.18%)
Substitutions (X): 13365/13714 detected (97.46%)

Results table:
```

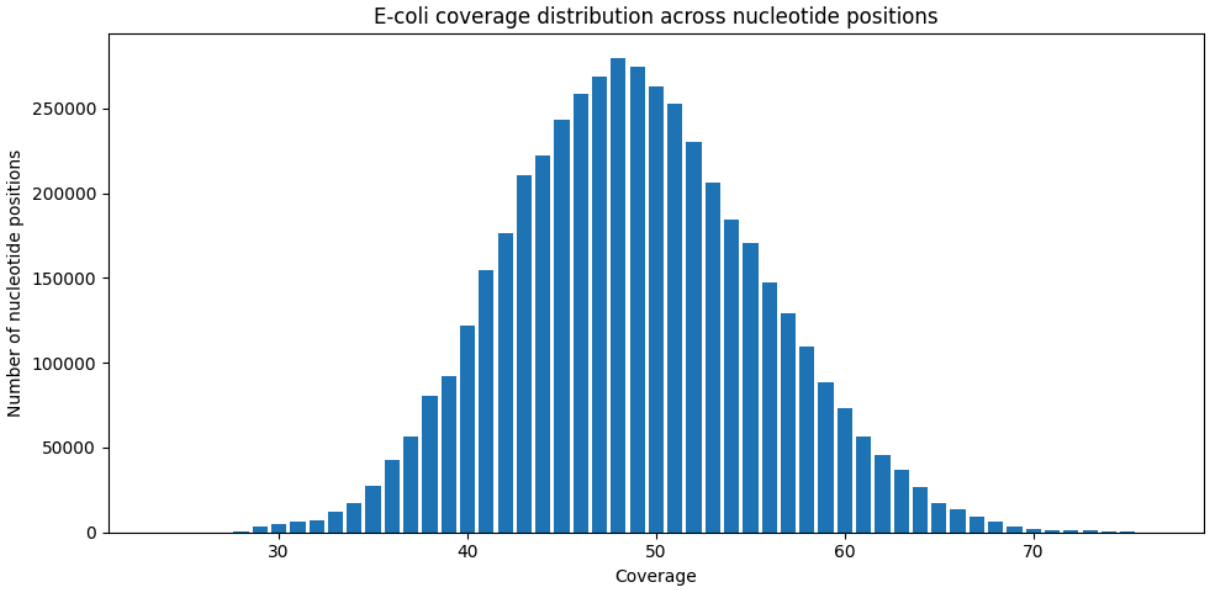
	mutation_name	mutation_type	detected	total	detected_percent
0	Deletions	D	1918	2871	66.81
1	Insertions	I	1779	2908	61.18
2	Substitutions	X	13365	13714	97.46



ecoli coverage distribution

In [7]:
```
base_path = r"C:\VSC\bioinf\ecoli"

pileup_path = base_path + r"\pileup.txt"

# pileup format:
# chrom, position, reference base, coverage, read bases, qualities
df = pd.read_csv(
    pileup_path,
    sep="\t",
    header=None,
    names=["chrom", "position", "ref", "coverage", "bases", "quals"]
)
```

```
coverage_counts = (
    df["coverage"]
    .value_counts()
    .sort_index()
)

plt.figure(figsize=(10, 5))

plt.bar(
    coverage_counts.index,
    coverage_counts.values
)

plt.xlabel("Coverage")
plt.ylabel("Number of nucleotide positions")
plt.title("E-coli coverage distribution across nucleotide positions")

plt.tight_layout()
plt.show()
```



----------LAMBDA (support >= 2, freq >= 70%)

In [10]:
```
base_path = r"C:\VSC\bioinf\lambda"

solution_file = base_path + r"\data\variants.csv"
truth_file = base_path + r"\stmpileup_var.csv"

cols = ["operation", "position", "base"]

solution = pd.read_csv(solution_file, header=None, names=cols)
truth = pd.read_csv(truth_file, header=None, names=cols)

solution["key"] = (
    solution["position"].astype(str) + "_" +
    solution["operation"] + "_" +
    solution["base"]
)
```

```
truth["key"] = (
    truth["position"].astype(str) + "_" +
    truth["operation"] + "_" +
    truth["base"]
)

solution_keys = set(solution["key"])
truth_keys = set(truth["key"])

matches = solution_keys.intersection(truth_keys)

num_matches = len(matches)
total_truth = len(truth_keys)

coverage = num_matches / total_truth * 100 if total_truth else 0

print(f"Lambda <> samtools coverage\n")
print(f"Matches found    : {num_matches}")
print(f"stmp variants    : {total_truth}")
print(f"Coverage of stmp : {coverage:.2f}%")
```

```
Lambda <> samtools coverage

Matches found    : 129
stmp variants    : 164
Coverage of stmp : 78.66%
```

stmp + off-by-one metrics

In [11]:
```
lookup = {}

for idx, row in solution.iterrows():

    key = (row["operation"], row["base"], row["position"])

    if key not in lookup:
        lookup[key] = []

    lookup[key].append(idx)

used_solution_indices = set()

tp = 0  # true positives
fn = 0  # false negatives

for _, trow in truth.iterrows():

    found = False

    t_op = trow["operation"]
    t_pos = trow["position"]
    t_base = trow["base"]

    key = (t_op, t_base, t_pos)

    if key in lookup:
```

```
        for s_idx in lookup[key]:

            if s_idx not in used_solution_indices:
                tp += 1
                used_solution_indices.add(s_idx)
                found = True
                break

    if not found:
        fn += 1

fp = len(solution) - len(used_solution_indices)

precision = tp / (tp + fp) if (tp + fp) > 0 else 0
recall = tp / (tp + fn) if (tp + fn) > 0 else 0
f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else

total = tp + fp + fn
accuracy = tp / total if total > 0 else 0

# convert to percentages
precision *= 100
recall *= 100
f1 *= 100
accuracy *= 100

print(f"Lambda <> samtools metrics\n")

print("\n================ CONFUSION MATRIX ================\n")

print("                   Truth")
print("                Present   Absent")
print(f"Pred Present    {tp:<8}  {fp}")
print(f"Pred Absent     {fn:<8}  -")

print("\n================ STATS ================\n")

print(f"Accuracy  : {accuracy:.2f}%")
print(f"Precision : {precision:.2f}%")
print(f"Recall    : {recall:.2f}%")
print(f"F1 Score  : {f1:.2f}%")
```

```
Lambda <> samtools metrics


================ CONFUSION MATRIX ================

                   Truth
                Present   Absent
Pred Present    129       96
Pred Absent     35        -

================ STATS ================

Accuracy  : 49.62%
Precision : 57.33%
Recall    : 78.66%
F1 Score  : 66.32%
```

ground truth matches

In [12]:
```python
solution_file = r"C:\VSC\bioinf\lambda\data\variants.csv"
truth_file = r"C:\VSC\bioinf\lambda\data\lambda_mutated.csv"

cols = ["operation", "position", "base"]

solution = pd.read_csv(solution_file, header=None, names=cols)
truth = pd.read_csv(truth_file, header=None, names=cols)

solution["key"] = (
    solution["position"].astype(str) + "_" +
    solution["operation"] + "_" +
    solution["base"]
)

truth["key"] = (
    truth["position"].astype(str) + "_" +
    truth["operation"] + "_" +
    truth["base"]
)

solution_keys = set(solution["key"])
truth_keys = set(truth["key"])

matches = solution_keys.intersection(truth_keys)

num_matches = len(matches)
total_truth = len(truth_keys)

coverage = num_matches / total_truth * 100 if total_truth else 0

print(f"Lambda <> ground truth coverage\n")
print(f"Matches found      : {num_matches}")
print(f"Ground truth       : {total_truth}")
print(f"Coverage of ground : {coverage:.2f}%")
```

```
Lambda <> ground truth coverage

Matches found      : 131
Ground truth       : 203
Coverage of ground : 64.53%
```

ground truth + off-by-one matches

In [13]:
```python
lookup = {}

for idx, row in solution.iterrows():

    key = (row["operation"], row["base"], row["position"])

    if key not in lookup:
        lookup[key] = []

    lookup[key].append(idx)

exact_matches = 0
off_by_one_matches = 0

used_solution_indices = set()

for _, trow in truth.iterrows():

    t_op = trow["operation"]
    t_pos = trow["position"]
    t_base = trow["base"]

    found = False

    exact_key = (t_op, t_base, t_pos)

    if exact_key in lookup:

        for s_idx in lookup[exact_key]:

            if s_idx not in used_solution_indices:
                exact_matches += 1
                used_solution_indices.add(s_idx)
                found = True
                break

    if not found:

        for offset in (-1, 1):

            off_key = (t_op, t_base, t_pos + offset)

            if off_key in lookup:

                for s_idx in lookup[off_key]:

                    if s_idx not in used_solution_indices:
                        off_by_one_matches += 1
                        used_solution_indices.add(s_idx)
```

```
                        found = True
                        break

            if found:
                break

total_matched = exact_matches + off_by_one_matches
coverage = (total_matched / len(truth) * 100) if len(truth) else 0

print(f"Lambda + off-by-one matches <> ground truth coverage\n")
print(f"Exact matches        : {exact_matches}")
print(f"Off-by-one matches   : {off_by_one_matches}")
print(f"Total matched        : {total_matched}")
print(f"Ground truth total   : {len(truth)}")
print(f"Coverage             : {coverage:.2f}%")
```

```
Lambda + off-by-one matches <> ground truth coverage

Exact matches        : 131
Off-by-one matches   : 33
Total matched        : 164
Ground truth total   : 203
Coverage             : 80.79%
```

ground truth + off-by-one metrics

In [14]:

```
lookup = {}

for idx, row in solution.iterrows():

    key = (row["operation"], row["base"], row["position"])

    if key not in lookup:
        lookup[key] = []

    lookup[key].append(idx)

used_solution_indices = set()

tp = 0  # true positives
fn = 0  # false negatives

for _, trow in truth.iterrows():

    found = False

    t_op = trow["operation"]
    t_pos = trow["position"]
    t_base = trow["base"]

    positions_to_check = [t_pos, t_pos - 1, t_pos + 1]

    for pos in positions_to_check:

        key = (t_op, t_base, pos)

        if key in lookup:
```

```
            for s_idx in lookup[key]:

                if s_idx not in used_solution_indices:
                    tp += 1
                    used_solution_indices.add(s_idx)
                    found = True
                    break

        if found:
            break

    if not found:
        fn += 1

fp = len(solution) - len(used_solution_indices)

precision = tp / (tp + fp) if (tp + fp) > 0 else 0
recall = tp / (tp + fn) if (tp + fn) > 0 else 0
f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else

total = tp + fp + fn
accuracy = tp / total if total > 0 else 0

# convert to percentages
precision *= 100
recall *= 100
f1 *= 100
accuracy *= 100

print(f"Lambda + off-by-one matches <> ground truth metrics\n")

print("\n================ CONFUSION MATRIX ================\n")

print("                   Truth")
print("               Present   Absent")
print(f"Pred Present    {tp:<8}  {fp}")
print(f"Pred Absent     {fn:<8}  -")

print("\n================ STATS ================\n")

print(f"Accuracy  : {accuracy:.2f}%")
print(f"Precision : {precision:.2f}%")
print(f"Recall    : {recall:.2f}%")
print(f"F1 Score  : {f1:.2f}%")
```

```
Lambda + off-by-one matches <> ground truth metrics


================ CONFUSION MATRIX ================

                   Truth
                Present   Absent
Pred Present    164       61
Pred Absent     39        -

================ STATS ================

Accuracy  : 62.12%
Precision : 72.89%
Recall    : 80.79%
F1 Score  : 76.64%
```

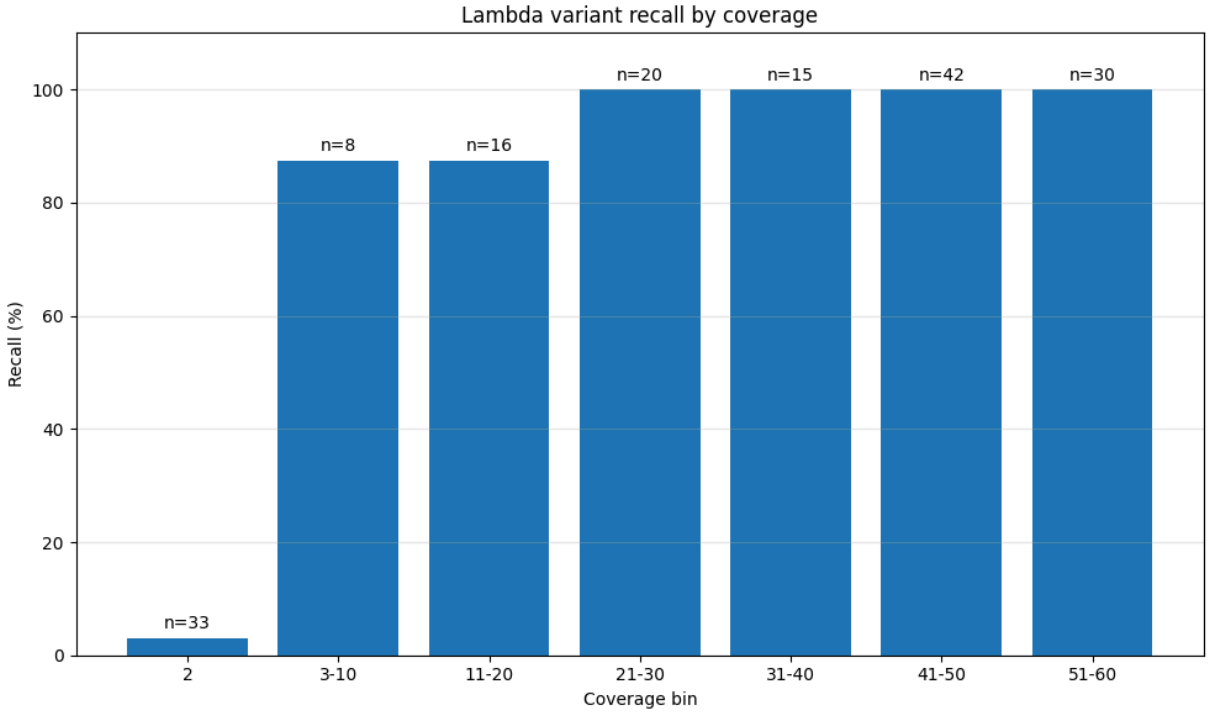
stmp groups

In [15]:
```python
base_path = r"C:\VSC\bioinf\lambda"

pileup_file = base_path + r"\pileup.txt"
solution_file = base_path + r"\data\variants.csv"
truth_file = base_path + r"\stmpileup_var.csv"

cols = ["operation", "position", "base"]

# ============================================================
# LOAD COVERAGE FROM PILEUP
# ============================================================

coverage_dict = {}

with open(pileup_file) as f:
    for line in f:
        fields = line.strip().split()

        if len(fields) < 4:
            continue

        pos = int(fields[1]) - 1   # mpileup je 1-based, CSV je 0-based
        cov = int(fields[3])

        coverage_dict[pos] = cov

# ============================================================
# LOAD VARIANT FILES
# ============================================================

solution = pd.read_csv(solution_file, header=None, names=cols)
truth = pd.read_csv(truth_file, header=None, names=cols)

# ============================================================
# CREATE KEYS
# ============================================================

solution["key"] = (
```

```
    solution["position"].astype(str) + "_" +
    solution["operation"] + "_" +
    solution["base"]
)

truth["key"] = (
    truth["position"].astype(str) + "_" +
    truth["operation"] + "_" +
    truth["base"]
)

solution_keys = set(solution["key"])

# ============================================================
# MATCH TRUTH VARIANTS WITH COVERAGE
# ============================================================

records = []

for _, row in truth.iterrows():

    pos = int(row["position"])
    cov = coverage_dict.get(pos, 0)

    matched = row["key"] in solution_keys

    records.append({
        "position": pos,
        "coverage": cov,
        "matched": int(matched)
    })

results = pd.DataFrame(records)

# ============================================================
# COVERAGE BINS
# ============================================================

bins = [1, 2, 10, 20, 30, 40, 50, 60]

labels = [
    "2",
    "3-10",
    "11-20",
    "21-30",
    "31-40",
    "41-50",
    "51-60",
]

results["coverage_bin"] = pd.cut(
    results["coverage"],
    bins=bins,
    labels=labels,
    include_lowest=True
)
```

```
# ============================================================
# STATISTICS BY COVERAGE BIN
# ============================================================

stats = (
    results
    .groupby("coverage_bin", observed=False)
    .agg(
        total=("matched", "count"),
        matches=("matched", "sum")
    )
    .reset_index()
)

stats["recall"] = stats["matches"] / stats["total"] * 100
stats["recall"] = stats["recall"].fillna(0)

print(stats)

# ============================================================
# PLOT
# ============================================================

plt.figure(figsize=(10, 6))

bars = plt.bar(
    stats["coverage_bin"].astype(str),
    stats["recall"]
)

for bar, total in zip(bars, stats["total"]):

    height = bar.get_height()

    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height + 1,
        f"n={total}",
        ha="center",
        va="bottom"
    )

plt.xlabel("Coverage bin")
plt.ylabel("Recall (%)")
plt.title("Lambda variant recall by coverage")

plt.ylim(0, 110)
plt.grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.show()
```

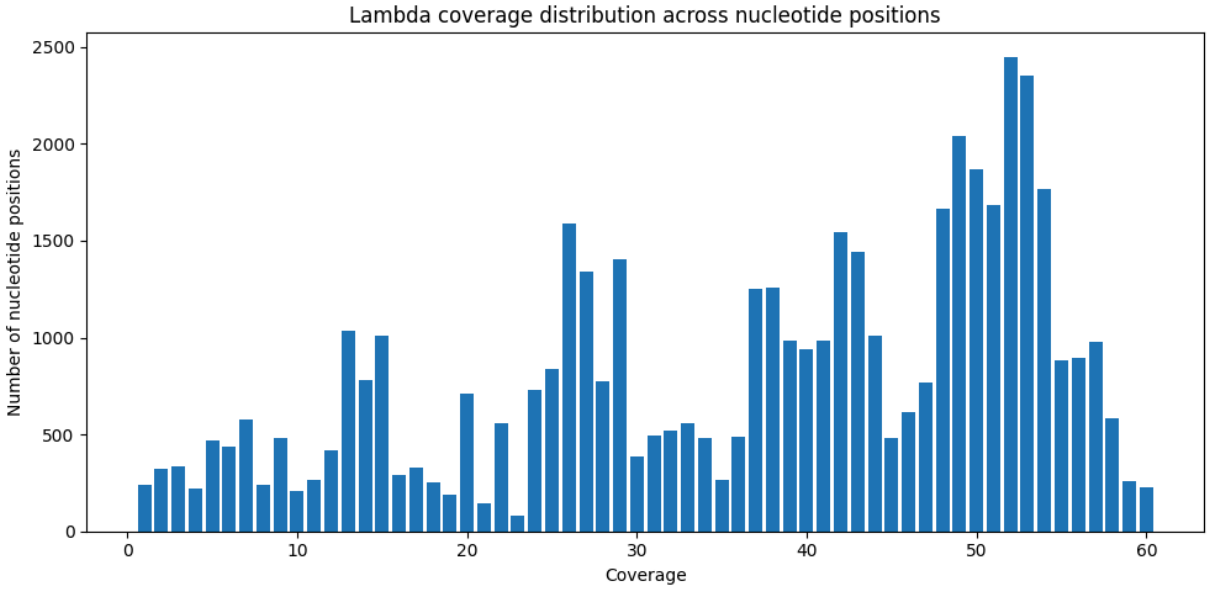
	coverage_bin	total	matches	recall
0	2	33	1	3.030303
1	3-10	8	7	87.500000
2	11-20	16	14	87.500000
3	21-30	20	20	100.000000
4	31-40	15	15	100.000000
5	41-50	42	42	100.000000
6	51-60	30	30	100.000000



lambda coverage distribution

In [6]:
```
base_path = r"C:\VSC\bioinf\lambda"

pileup_path = base_path + r"\pileup.txt"

# pileup format:
# chrom, position, reference base, coverage, read bases, qualities
df = pd.read_csv(
    pileup_path,
    sep="\t",
    header=None,
    names=["chrom", "position", "ref", "coverage", "bases", "quals"]
)

coverage_counts = (
    df["coverage"]
    .value_counts()
    .sort_index()
)

plt.figure(figsize=(10, 5))

plt.bar(
    coverage_counts.index,
    coverage_counts.values
```

```
)

plt.xlabel("Coverage")
plt.ylabel("Number of nucleotide positions")
plt.title("Lambda coverage distribution across nucleotide positions")

plt.tight_layout()
plt.show()
```



In [ ]: